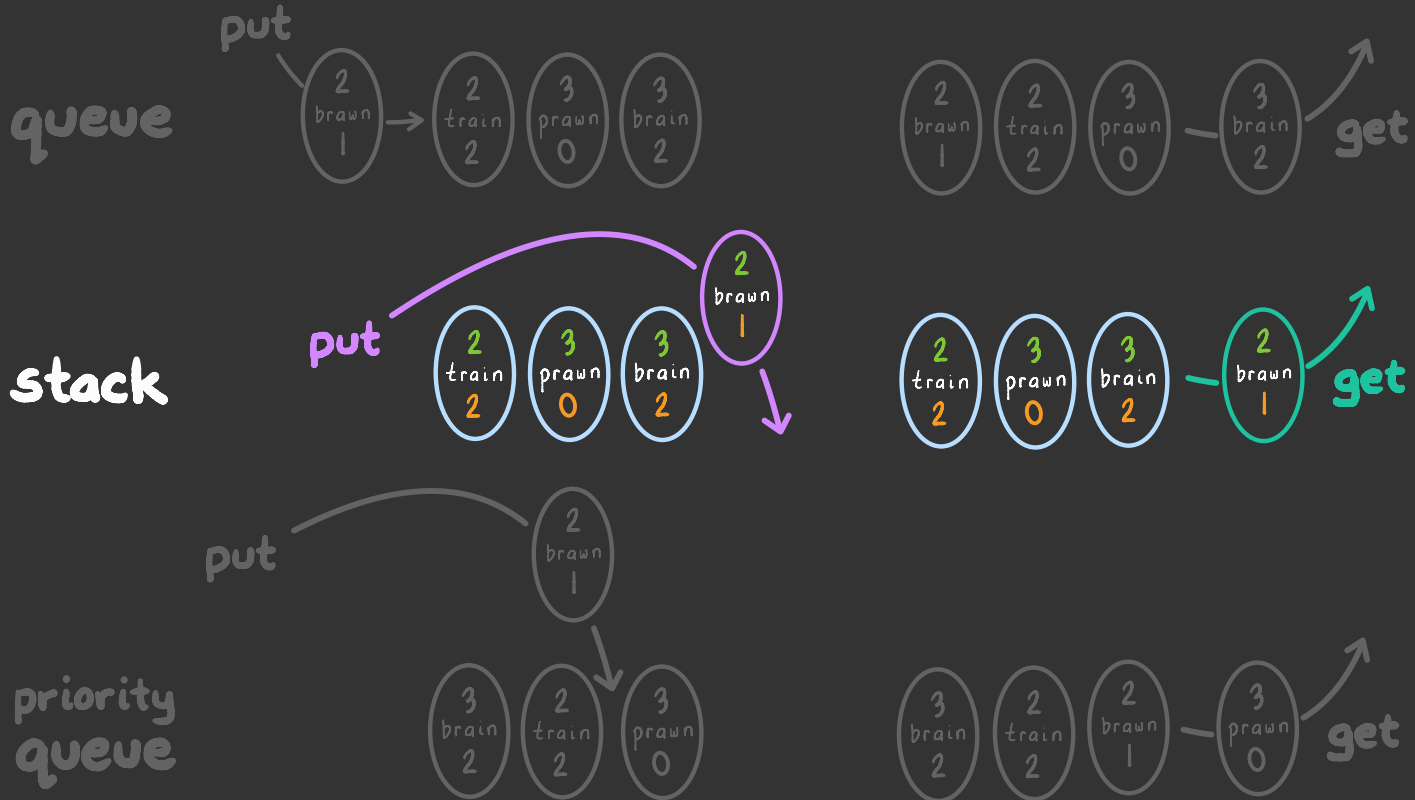


depth-first search

CSCI
270

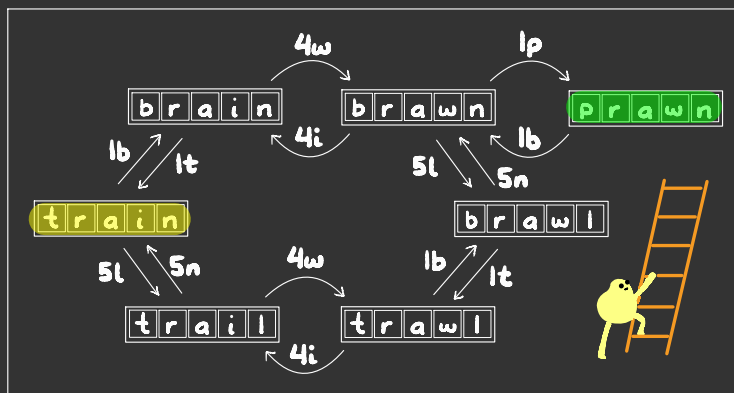
there's one more container!



0
train
2



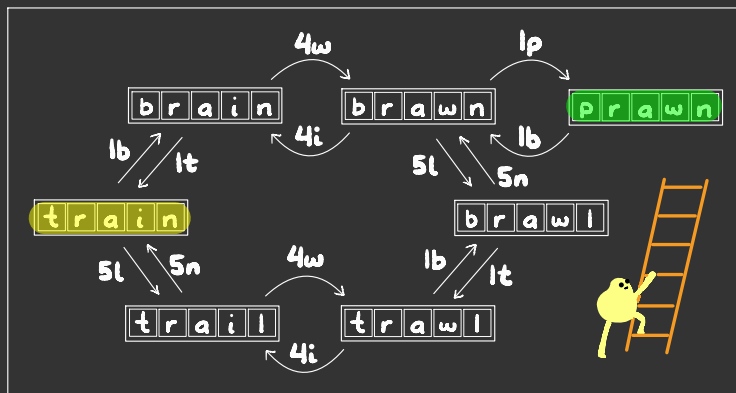
0
train
2

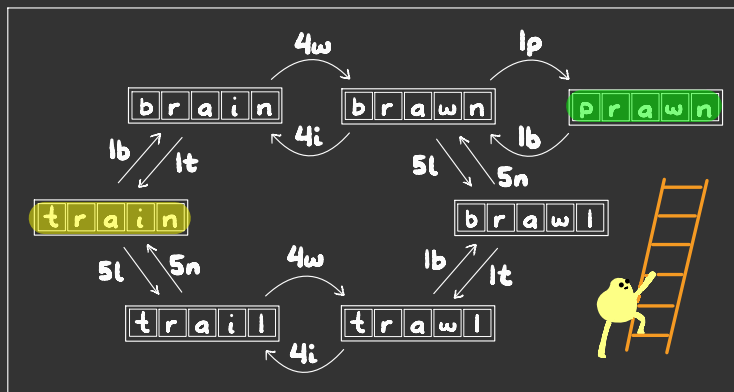
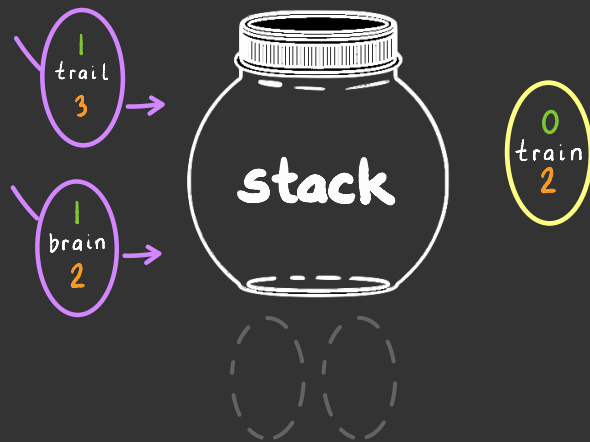
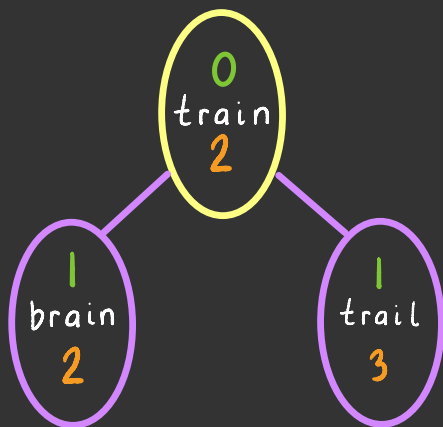


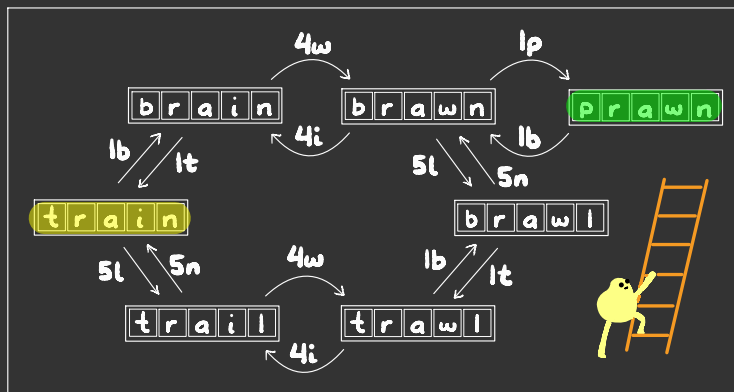
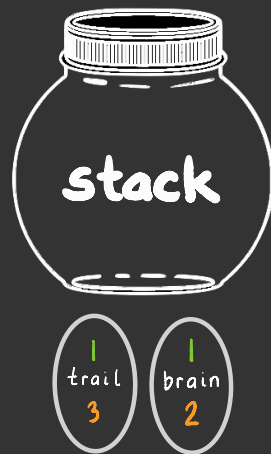
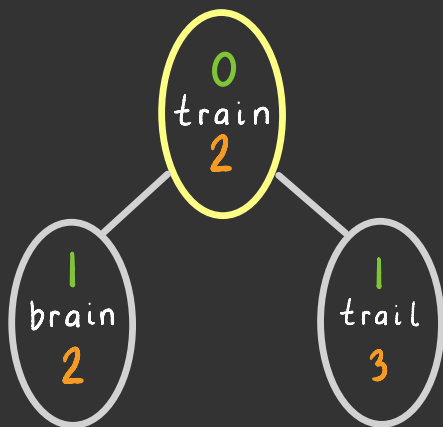
0
train
2

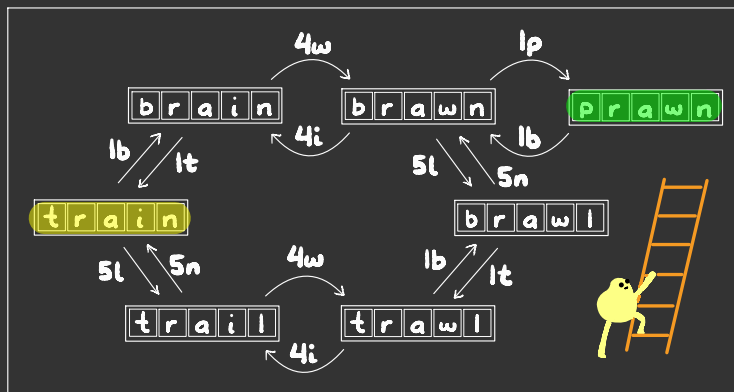
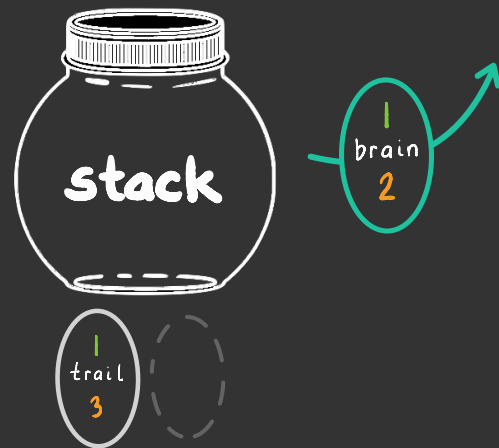
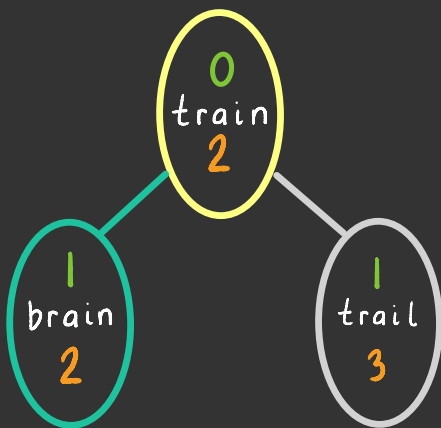
stack

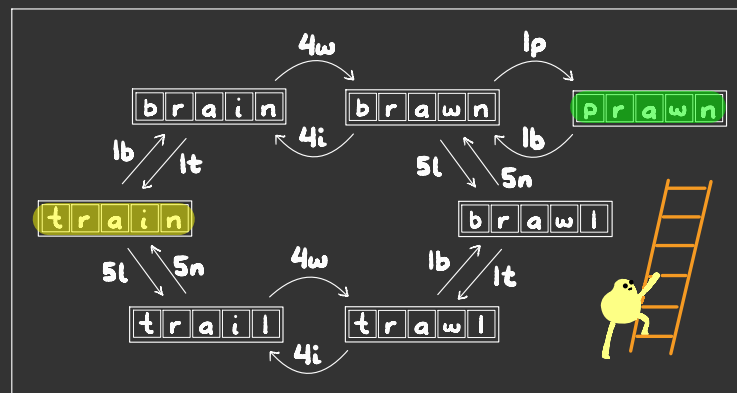
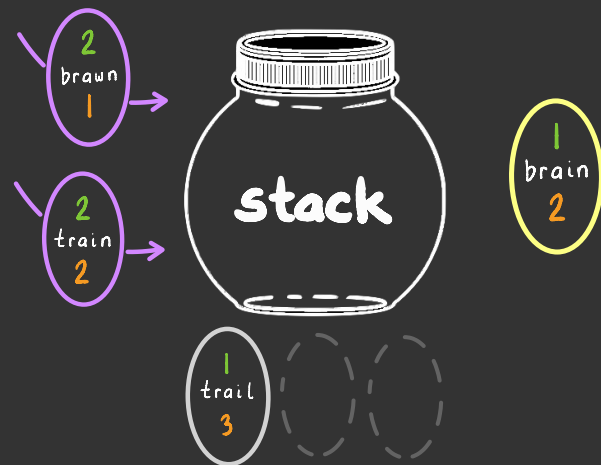
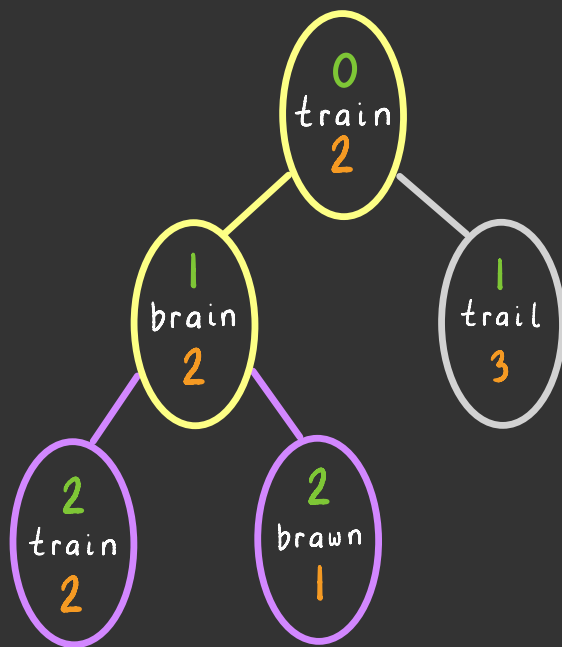
0
train
2

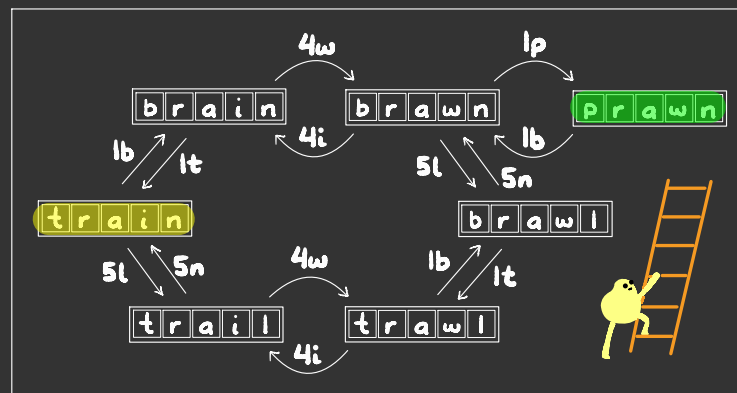
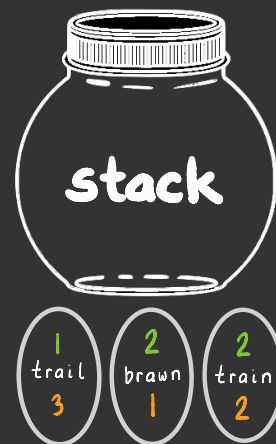
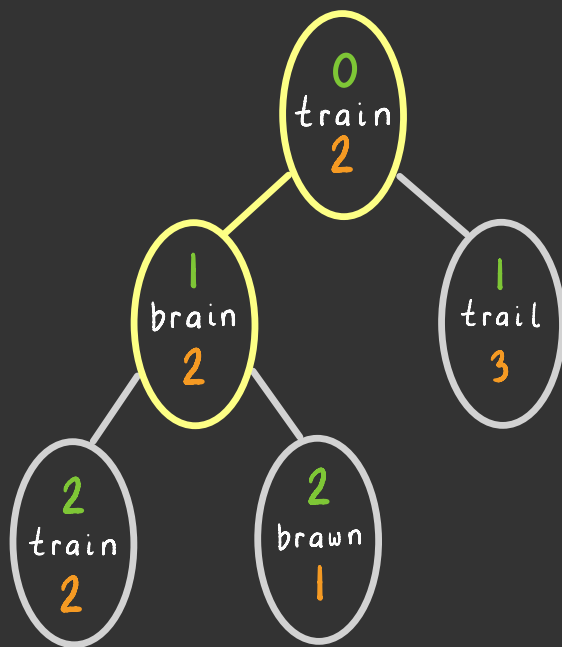


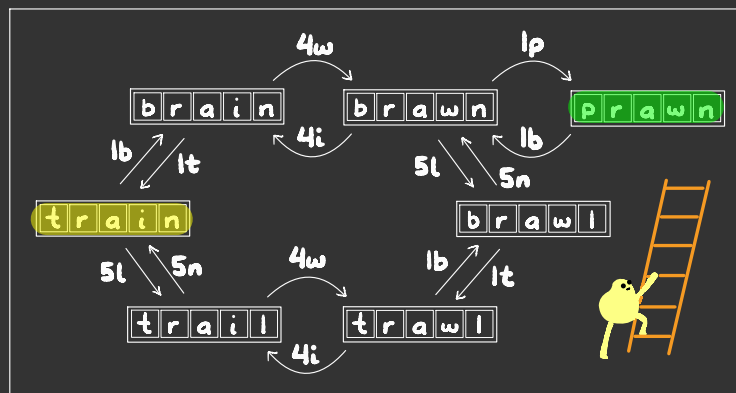
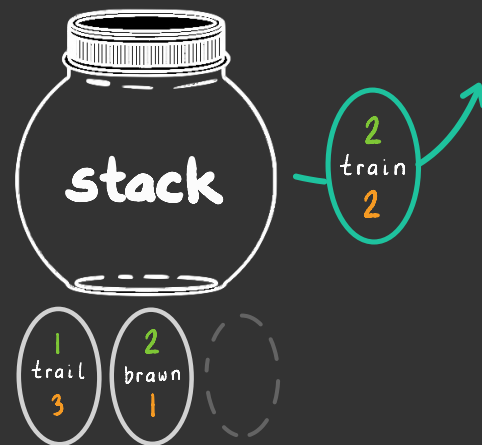
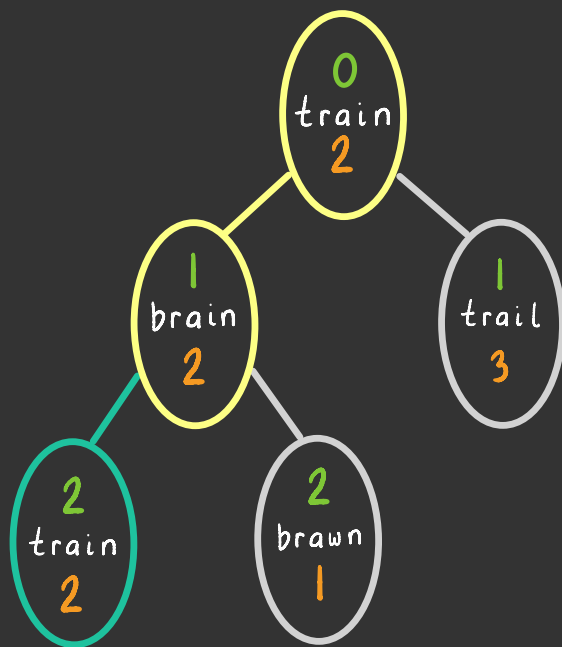


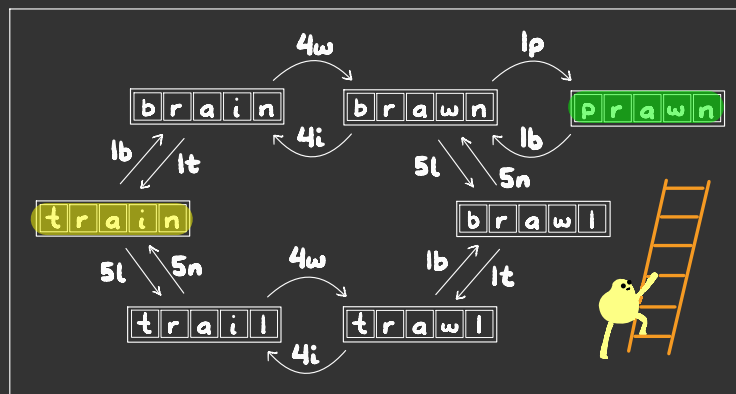
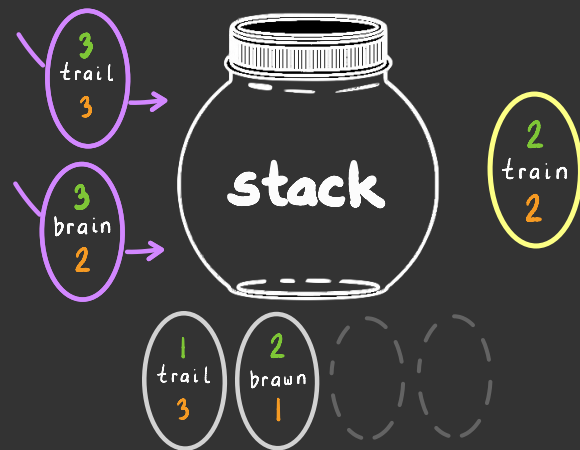
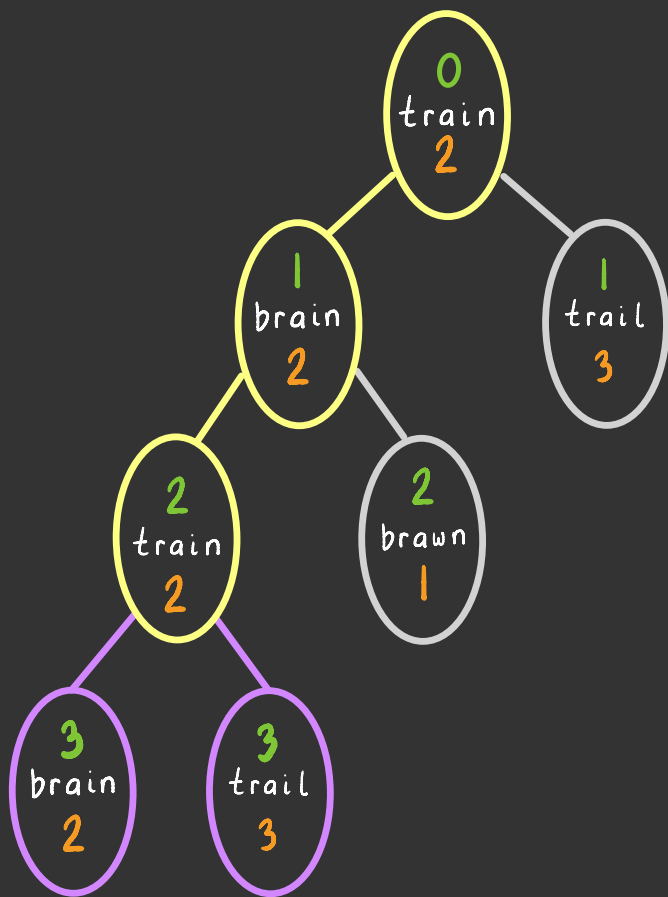


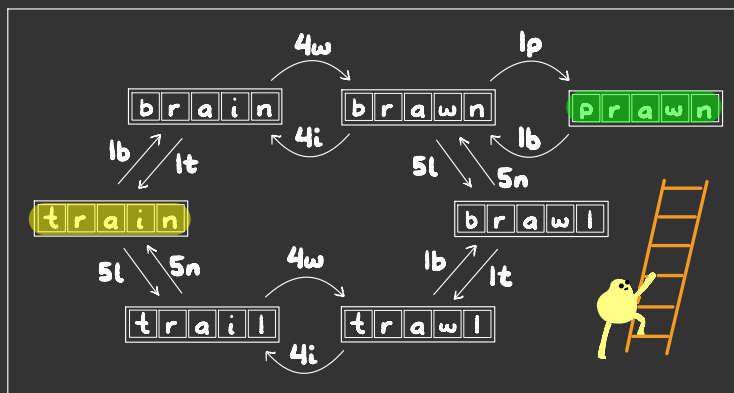
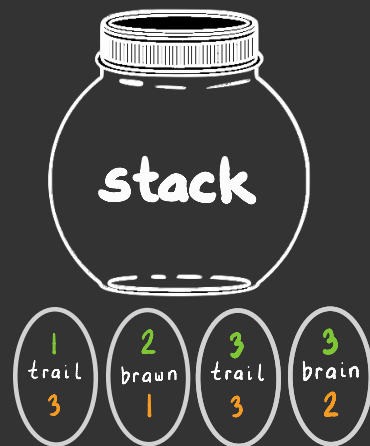
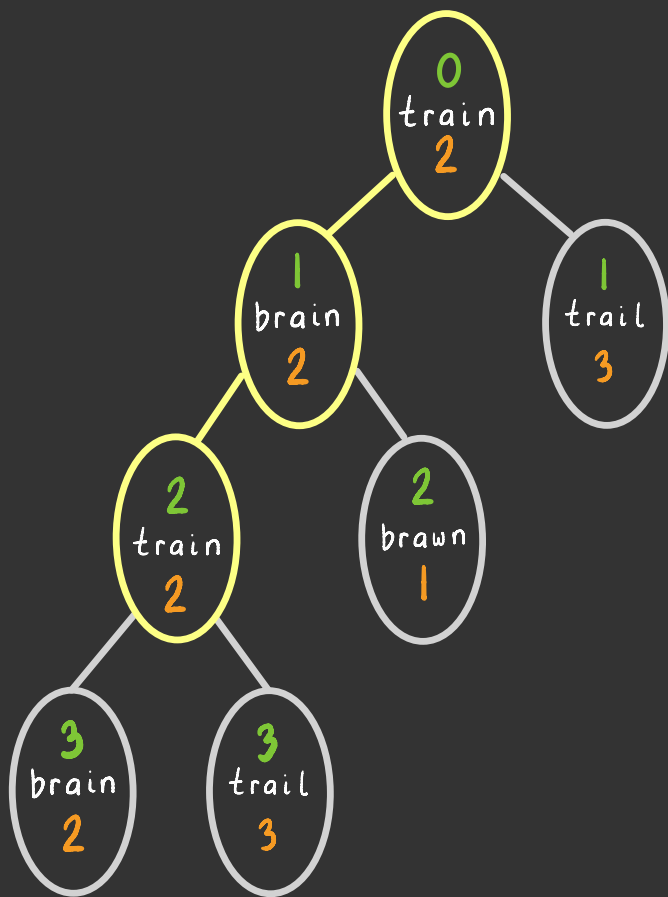


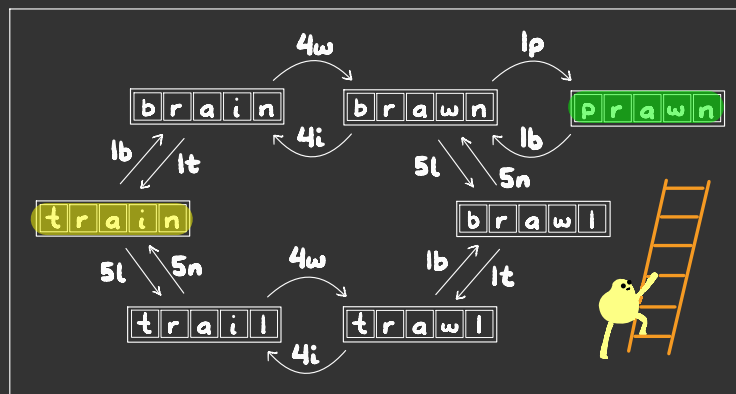
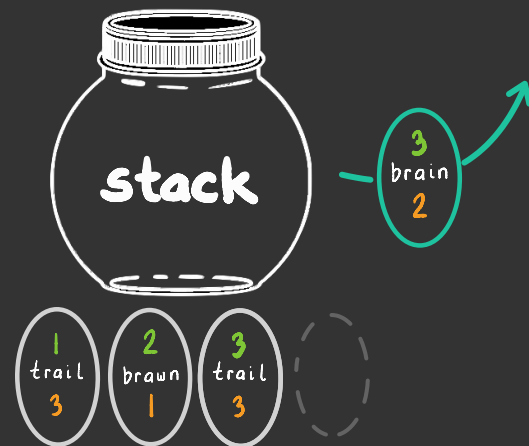
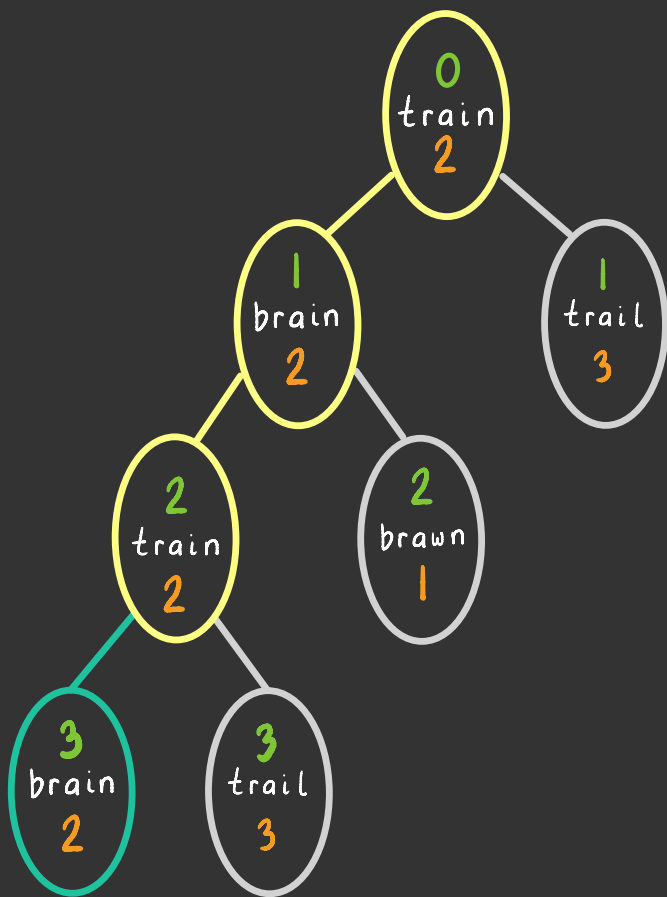




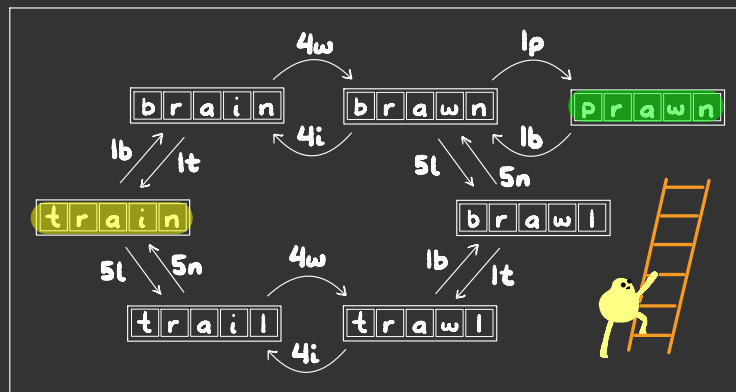
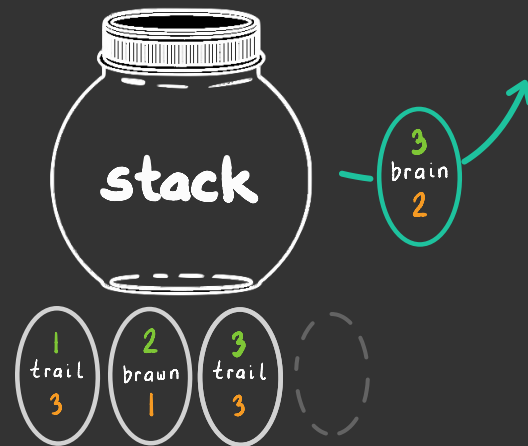
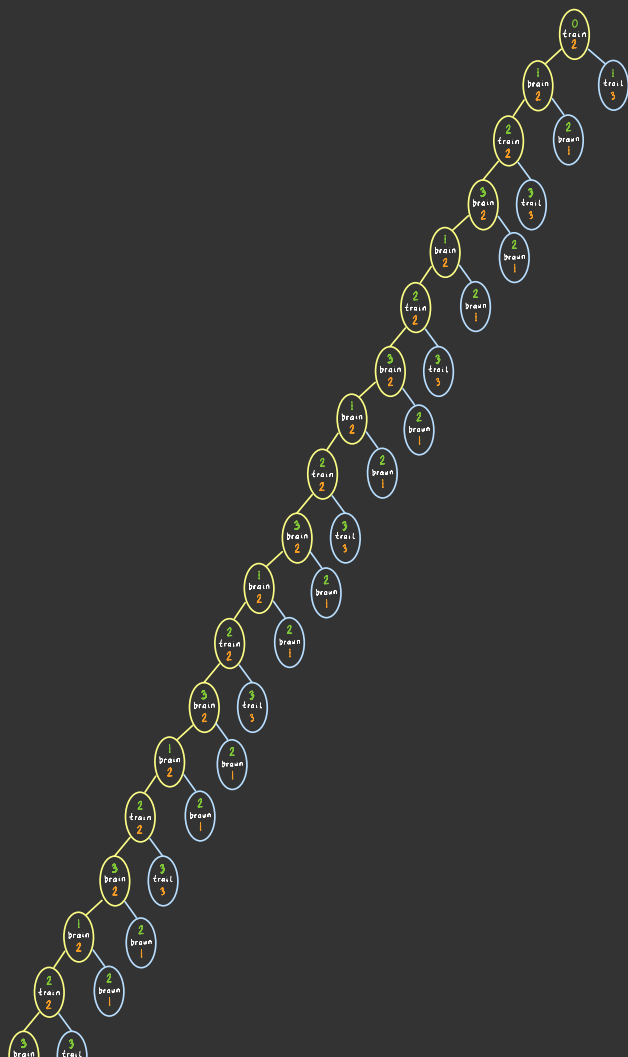








a few steps later



search $(M = (Q, \Sigma, \Delta, q_0, F, \omega), H)$:

- ▶ `container = new Container()`
- ▶ `container.put( $\langle q_0, 0, H(q_0) \rangle$ )`
- ▶ repeat:
 - ▶ if `container.empty()` then return ∞
 - ▶ `n = container.get()`
 - ▶ if $q(n) \in F$ then return $g(n)$
 - ▶ for each transition $(q(n), \sigma, q') \in \Delta$:
 - `successor =  $\langle q', g(n) + \omega(q(n), \sigma, q'), H(q') \rangle$`
 - `container.put(successor)`

"visit"
a node

if we use a



as the **container**,
the algorithm is called
depth-first
search (dfs)



breadth-first
search (bfs)

if all transitions
have the same
weight, then
bfs is optimal



uniform-cost
search (ucs)

if all transitions
have non-negative
weight, then
ucs is optimal



depth-first
search (dfs)

non-optimal,
but has its
charms